

# 설계서 작성 지침(GPT)

## D. 클래스 설계서를 어떻게 채우면 좋은가

제가 보기에는 **“기능 기준 클래스” + “DB 기준 클래스”**를 같이 적는 방식이 가장 좋습니다.

### 1) 설계 클래스 목록은 이렇게 묶는 것이 좋습니다

#### 컨트롤러 계층 10

- LandingController (랜딩페이지) - 동렬
- MainController (메인페이지) - 찬용
- AuthController (회원 페이지) - 찬용
- FindAccountController (회원찾기 페이지, 일단 보류) - 찬용
- DashboardController (대시보드 페이지) - 현규
- MediaController (미디어 페이지) - 동렬
- HistoryController (펫관리 페이지) - 도현
- SettingsController (환경설정 페이지) - 도현
- AdminController (관리자 페이지) - 현규
- SimulationController (시뮬레이션 페이지) - 도현

#### 서비스 계층 13

- AuthService
- MainService
- FindAccountService (추가)
- DashboardService
- MediaService
- HistoryService
- PetService
- SettingsService

- SimulationService
- EventEvaluationService
- AlertService
- ActionService
- AdminService

## 엔티티 계층 20

- User
- Pet
- PetBaselineSetting
- Device
- UserNotificationSetting
- EventThresholdSetting
- WearableLog
- FeederLog
- WaterLog
- CameraLog
- DeviceAnalysisLog
- State
- EventLog
- RiskScoreLog
- AlertLog
- ActionLog
- MediaLog
- EnvironmentLog
- BehaviorCode
- EventCode

## DTO 계층 7 (이후 수정 필요)

- LoginRequest / LoginResponse

- SignupRequest
- ~~DashboardMainResponse~~-(Thymeleaf Model로 처리)
- ~~HistoryAlertResponse~~-(Thymeleaf Model로 처리)
- ~~HistoryActionResponse~~-(Thymeleaf Model로 처리)
- PetCreateRequest / PetUpdateRequest
- ThresholdSettingRequest
- SimulationRunResult

이렇게 가면 문서가 훨씬 정리됩니다.

---

## E. 클래스 설계서 작성 방식 수정안

교수님 양식의 "설계 클래스 목록"에는 **모든 엔티티를 다 적기보다**,  
우선은 **핵심 동작 클래스**를 먼저 적는 게 좋습니다.

추천 우선순위는 이렇습니다.

- Controller 10개 내외
- Service 8~10개
- 핵심 Entity 8~12개
- DTO는 정말 필요한 것만 대표로

왜냐하면 모든 DTO까지 다 적기 시작하면 문서가 너무 커지기 때문입니다.

## 추천 핵심 클래스 1차 세트

- AuthController / AuthService / User
- MainController / MainService
- DashboardController / DashboardService
- HistoryController / HistoryService
- SettingsController / PetService / SettingsService
- SimulationController / SimulationService / EventEvaluationService
- State / EventLog / AlertLog / ActionLog / MediaLog / Pet / EventThresholdSetting

이 정도면 교수님 문서용으로도 충분히 "설계했다"는 느낌이 납니다.

---

## F. 실제로 적을 때의 예시 형식

예를 들어 `SimulationService` 는 이렇게 쓰면 됩니다.

### 설계 클래스 목록

- 설계 클래스 ID: CLS-SVC-007
- 설계 클래스명: SimulationService
- 클래스 설명: 상태 데이터 기반 시뮬레이션 실행 및 이벤트/알림/조치 생성
- 관련 요구사항 ID: sim-001, sim-002

### 클래스 정의

- 클래스명: SimulationService
- Annotation: `@Service`

### 속성

- stateRepository : StateRepository
- eventLogRepository : EventLogRepository
- alertLogRepository : AlertLogRepository
- actionLogRepository : ActionLogRepository
- eventEvaluationService : EventEvaluationService

### 메서드

- `getStateList(Long petId)` : 상태 데이터 목록 조회
- `runSimulation(List<Long> stateIds)` : 시뮬레이션 실행
- `createEvent(State state)` : 이벤트 생성
- `createAlert(EventLog event)` : 알림 생성
- `createAction(AlertLog alert)` : 조치 생성

## G. DB설계 기준 수정 권고안

클래스 설계를 안정적으로 쓰려면 DB에서 아래는 먼저 손보는 게 좋습니다.

- ◆ `state`

- `recorded_at` 추가
- `processed` 추가
- `created_at` 추가
- ◆ `alert_log`
  - `alert_status` 는 boolean 대신 enum 권장  
예: `PENDING` , `DISPLAYED` , `FAILED` , `ACKNOWLEDGED`
- ◆ `action_log`
  - `action_status` 도 boolean 대신 enum 권장  
예: `REQUESTED` , `SUCCESS` , `FAILED`
- ◆ `risk_score_log`
  - 현재 컬럼 줄 구성이 다소 흐트러져 보여서 정리 필요
  - `event_log_id` , `action_log_id` 를 둘 다 들지, 아니면 `state_id` 중심으로 같지 결정 필요
- ◆ `pets`
  - `device_id` 단일 FK가 정말 맞는지 재검토 필요  
한 반려동물에 카메라/급수기/급여기/웨어러블이 모두 연결되면 단일 `device_id` 는 좁습니다.

## H. API설계 기준 수정 권고안

- ◆ `dash-002`
  - 한 줄의 `GET/POST` 를 분리  
예: `GET /dashboard/stream``POST /dashboard/stream/ptz`
- ◆ `history-001`
  - URL을 `/history/alerts` 로 통일
- ◆ `media-001`
  - `page-008` 병합 표현 제거
  - `media-001` 을 독립 행으로 정리
- ◆ 수정/삭제 URL
  - 지금 방식도 동작은 가능하지만, 클래스 설계서 기준으로는
    - `GET /settings/pets/{petId}/edit`

- `POST /settings/pets/{petId}/edit`

또는

- `GET /settings/pets/{petId}`

- `POST /settings/pets/{petId}`

처럼 일관되게 맞추는 편이 깔끔합니다.

이유는 컨트롤러 메서드명과 DTO 구성이 더 자연스러워지기 때문입니다.